Nhóm 04 - T44 - AI Agent &Tool-Use trong Microservices

Môn học: INT4425 — Dịch vụ Web · Giảng viên: Đặng Ngọc Hùng
Phiên bản: V1 · Đợt vấn đáp: Đợt 1 · Hoạt động 1 — Nghiên cứu topic

Thành viên & Phân công

STT	Họ và tên	MSSV	Phần đảm nhận	Đóng góp
1	Nguyễn Lê Trúc Quỳnh	B25CHKH081	A2 (kiến trúc, Tool Gateway, Hình 1); A1 (bài toán); tổng hợp và trình bày báo cáo	Chính
2	Đỗ Nhật Minh	B25CHKH075	A1 (các mốc lịch sử); A2 (bước 2–3, agent loop); A3 (Klarna); A4 (ưu/nhược điểm)	Chính
3	Trần Khánh Hưng	B25CHKH069	A2 (bước 1, 7, guardrails); A3 (LinkedIn, Bảng 1); A4 (nên/không nên dùng); tài liệu tham khảo	Chính

A1. Vấn đề & Bối cảnh

Doanh nghiệp thương mại điện tử muốn dùng mô hình ngôn ngữ lớn (LLM) để chăm sóc khách hàng: tra cứu đơn, đổi địa chỉ giao hàng, xử lý hoàn tiền. Tuy nhiên LLM thuần chỉ sinh văn bản từ dữ liệu huấn luyện: không biết trạng thái đơn hàng hiện tại, không thực hiện được thao tác và có thể "bịa" chính sách. Dữ liệu và nghiệp vụ thật nằm trong các microservice có sẵn API. Bài toán đặt ra là **cho LLM gọi các API này như công cụ (tool)**, đồng thời **giới hạn và kiểm soát** được việc nó làm, vì một lời gọi sai có thể gây mất tiền hoặc lộ dữ liệu.

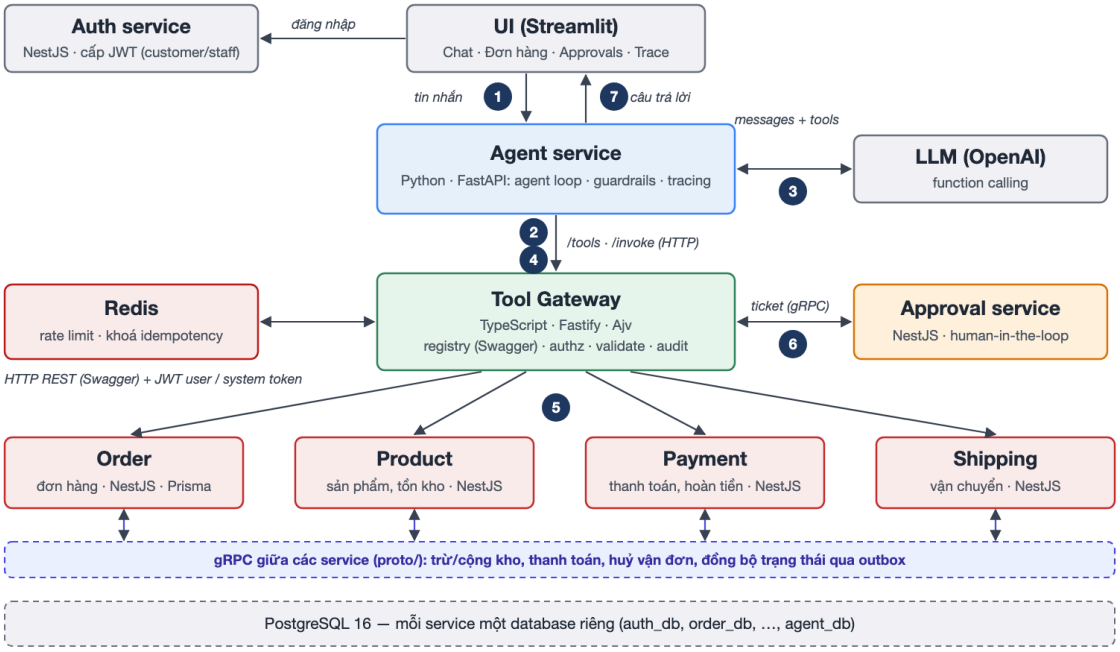
Các mốc chính hình thành kỹ thuật này:

- **10/2022** - Yao và cộng sự (Princeton, Google Research) công bố **ReAct**, cho LLM xen kẽ suy luận và hành động (gọi công cụ, quan sát kết quả) trong một vòng lặp [2].
- **02/2023** - Schick và cộng sự (Meta AI) công bố **Toolformer**, cho thấy LLM có thể tự học khi nào và cách gọi API [3]. Cùng tháng, Greshake và cộng sự chỉ ra **prompt injection gián tiếp**: chỉ thị độc hại nằm trong dữ liệu mà công cụ trả về [5].
- **06/2023** - OpenAI đưa **function calling** vào API: nhà phát triển mô tả hàm bằng JSON Schema, mô hình trả về tên hàm và tham số để ứng dụng thực thi [1].
- **08/2023** - OWASP công bố Top 10 rủi ro cho ứng dụng LLM, trong đó có *Prompt Injection* và *Excessive Agency* (agent được trao quá nhiều quyền), bản 2025 vẫn xếp *Prompt Injection* (LLM01) và *Excessive Agency* (LLM06) trong danh sách [6].
- **11/2024** - Anthropic công bố **Model Context Protocol (MCP)**, chuẩn mở để kết nối LLM với công cụ và nguồn dữ liệu [4].

A2. Cách hoạt động

Nhóm minh hoạ kỹ thuật bằng hệ thống **OpsPilot**: agent chăm sóc khách hàng gọi năm microservice nghiệp vụ (Auth, Order, Product, Payment, Shipping) thông qua một Tool Gateway. Agent và giao diện viết bằng Python; Tool Gateway và các microservice viết bằng TypeScript. Các thành phần chính:

Thành phần	Vai trò
LLM	Chọn công cụ và sinh tham số dựa trên câu hỏi và mô tả công cụ (function calling); không trực tiếp thực thi.
Agent	Điều phối vòng lặp: gửi hội thoại cho LLM, chuyển yêu cầu gọi công cụ tới Gateway, trả kết quả lại cho LLM; áp dụng guardrails và ghi vết.
Tool Gateway	Cổng duy nhất tới microservice (TypeScript, Fastify). Giữ **tool registry** sinh tự động từ đặc tả OpenAPI [7] của các service; phân quyền, kiểm tra tham số theo JSON Schema (Ajv), giới hạn tần suất và khoá idempotency trên Redis, ghi nhật ký.
Microservices	Thực hiện nghiệp vụ và tự kiểm tra quyền sở hữu dữ liệu. Mỗi service viết bằng NestJS, chạy độc lập với cơ sở dữ liệu riêng (PostgreSQL, Prisma); mở REST kèm đặc tả Swagger cho Gateway, còn giữa các service gọi nhau qua gRPC [11] (trừ kho, thanh toán, huỷ vận đơn, đồng bộ trạng thái).
Approval	Service NestJS giữ thao tác rủi ro cao ở trạng thái chờ cho tới khi nhân viên phê duyệt (human-in-the-loop); Gateway tạo và đọc yêu cầu qua gRPC.



Hình 1. Kiến trúc OpsPilot; số thứ tự tương ứng các bước xử lý bên dưới, khung nét đứt xanh là các lời gọi gRPC giữa microservice

Mỗi endpoint muốn cho agent dùng được gắn một decorator khai báo tên công cụ, mức rủi ro (thấp, trung bình, cao) và vai trò được phép; khai báo này nằm ngay

trong đặc tả Swagger do service sinh ra nên không lệch với code. Gateway đọc các khai báo này để dựng danh sách **20 công cụ**: 12 công cụ đọc, 7 công cụ ghi và 1 công cụ rủi ro cao (hoàn tiền). Một lượt hội thoại diễn ra như sau:

- **(1)** Người dùng gửi tin nhắn kèm token xác thực; Agent kiểm tra prompt injection trực tiếp trong tin nhắn.
- **(2)** Agent lấy từ Gateway danh sách công cụ **theo vai trò** của người dùng; công cụ ngoài quyền không bao giờ được gửi cho LLM.
- **(3)** Agent gửi hội thoại và danh sách công cụ cho LLM; LLM trả về câu trả lời hoặc yêu cầu gọi công cụ kèm tham số.
- **(4)** Agent chuyển yêu cầu gọi công cụ tới Gateway.
- **(5)** Gateway lần lượt kiểm tra quyền, kiểm tra tham số theo JSON Schema (Ajv), kiểm tra mã xác nhận do chính người dùng nhập (với đặt hàng, thanh toán, phê duyệt) và giới hạn tần suất (bộ đếm trên Redis), rồi gọi REST của microservice; thao tác ghi kèm khoá idempotency và được khoá trên Redis để không gửi trùng. Nếu cần, microservice gọi tiếp các service khác qua gRPC.
- **(6)** Với công cụ rủi ro cao, Gateway không thực thi mà tạo yêu cầu ở Approval (qua gRPC); khi nhân viên duyệt, Gateway mới thực thi thay mặt khách hàng.
- **(7)** Kết quả công cụ được quét chỉ thị độc hại, đánh dấu là dữ liệu không tin cậy rồi trả lại LLM; bước (3)–(7) lặp lại tối đa 6 lần hoặc 12.000 token. Câu trả lời cuối được che thông tin cá nhân không thuộc người dùng trước khi gửi đi.

A3. Ứng dụng thực tế

Klarna - trợ lý AI chăm sóc khách hàng (02/2024). Klarna, công ty thanh toán của Thụy Điển, phải xử lý khối lượng lớn yêu cầu hỗ trợ ở 23 thị trường. Công ty triển khai trợ lý AI xây dựng trên mô hình của OpenAI, xử lý các yêu cầu như hoàn tiền, đổi trả, thanh toán. Theo công bố của Klarna, sau một tháng trợ lý đã thực hiện 2,3 triệu cuộc hội thoại (hai phần ba tổng số), tương đương khối lượng của 700 nhân viên, thời gian giải quyết giảm từ 11 phút xuống dưới 2 phút và yêu cầu lặp lại giảm 25% [8]. Tuy nhiên đến năm 2025CEO Klarna thừa nhận việc ưu tiên chi phí đã làm chất lượng dịch vụ giảm và công ty đầu tư trở lại vào nhân viên hỗ trợ [10] - cho thấy agent cần đi kèm cơ chế chuyển cho con người.

LinkedIn - Hiring Assistant cho nhà tuyển dụng (10/2024). Nhà tuyển dụng thường mất nhiều giờ cho mỗi vị trí chỉ để tìm kiếm và sàng lọc hồ sơ. Hiring Assistant là AI agent đầu tiên của LinkedIn: từ yêu cầu tuyển dụng, agent tự tìm ứng viên phù hợp trong mạng lưới LinkedIn, đánh giá hồ sơ và đề xuất danh sách; nhà tuyển dụng phản hồi qua từng vòng để agent điều chỉnh kết quả và vẫn là người ra quyết định. Agent được triển khai cho nhóm khách hàng đầu tiên từ 10/2024 và mở rộng toàn cầu (tiếng Anh) từ 09/2025, với các khách hàng như AMD, Expedia Group, Microsoft và Siemens. Theo LinkedIn, người dùng sớm tiết kiệm hơn 4 giờ cho mỗi vị trí, phải xem ít hơn 62% hồ sơ và tỉ lệ chấp nhận InMail tăng 69% [9]. Điểm tương

đồng với OpsPilot: agent đảm nhận các bước tốn công, còn quyết định quan trọng vẫn do con người đưa ra.

Kết quả trên OpsPilot. Nhóm đánh giá hệ thống bằng 51 kịch bản (tra cứu, nhiều bước, vượt quyền, thao tác rủi ro, injection trực tiếp và gián tiếp…) với bốn cấu hình bảo vệ, dùng mô hình gpt-4o-mini:

Bảng 1. Kết quả đánh giá theo cấu hình bảo vệ

Chỉ số	Không bảo vệ	Lọc đầu vào	+ Quét kết quả	Đầy đủ
Hoàn thành tác vụ	84,3%	96,1%	100%	100%
Tấn công thành công	27,3%	18,2%	0%	0%
Chặn nhầm	0%	0%	0%	0%
Độ trễ p95 (giây)	2,9	2,8	5,7	10,2

A4. Đánh giá - Khi nào nên / Không nên dùng

Ưu điểm. Người dùng thao tác bằng ngôn ngữ tự nhiên trên dữ liệu thật thay vì nhận câu trả lời chung chung. Công cụ được sinh từ API có sẵn nên thêm chức năng mới chỉ cần khai báo trên endpoint, không sửa agent. Kiểm soát đặt ở Gateway và service nên vẫn có hiệu lực khi LLM bị đánh lừa: trong thử nghiệm, kể cả lúc agent bị injection gọi lệnh hoàn tiền, yêu cầu vẫn chỉ dừng ở trạng thái chờ duyệt.

Nhược điểm. Hành vi của LLM không tất định: với câu tường thuật như "Đơn 1003 đã được lấy hàng", gpt-4o-mini chỉ chọn đúng công cụ 3/9 lần. Không có bảo vệ, 27,3% kịch bản tấn công thành công; thêm bảo vệ lại làm độ trễ p95 tăng từ 2,9 lên 10,2 giây. Mỗi lượt tốn khoảng 4.600–5.200 token, và hệ thống phức tạp hơn nhiều so với một API thông thường.

Nên dùng khi	Không nên dùng khi
Yêu cầu đa dạng, diễn đạt tự do, cần kết hợp nhiều API trong một câu hỏi.	Quy trình cố định, ít bước - form hoặc nút bấm nhanh, rẻ và chắc chắn hơn.
Phần lớn thao tác là đọc; thao tác ghi có thể hoàn tác hoặc có người duyệt.	Thao tác không thể hoàn tác mà không có bước xác nhận hay phê duyệt.
API đã có phân quyền theo người dùng và có thể kiểm toán.	Cần độ trễ thấp (dưới 1 giây) hoặc kết quả tất định tuyệt đối.
Có người tiếp nhận khi agent không xử lý được.	Không có ngân sách cho kiểm thử, giám sát và chi phí token.

Kết luận & Bài học cá nhân

Nguyễn Lê Trúc Quỳnh: Qua quá trình triển khai cơ chế phân quyền tại Tool Gateway, em nhận thấy rằng an toàn của agent không chỉ phụ thuộc vào prompt. Ngay cả khi LLM bị prompt injection và cố gắng gọi lệnh hoàn tiền, yêu cầu vẫn được giữ ở trạng thái chờ phê duyệt. Vì vậy, khi xây dựng hệ thống thực tế, em sẽ coi agent là một client không đáng tin cậy, ưu tiên cấp quyền đọc trước, sử dụng khóa idempotency cho các thao tác ghi và yêu cầu con người phê duyệt những hành động có rủi ro cao.

Đỗ Nhật Minh: Kết quả thử nghiệm cho thấy năng lực của mô hình chưa phải yếu tố duy nhất quyết định độ chính xác của function calling. Với các câu tường thuật, GPT-4o-mini chỉ chọn đúng công cụ trong 3/9 lần thử. Điều này cho thấy tên gọi, mô tả và schema của tool có ảnh hưởng đáng kể đến kết quả. Trong công việc thực tế, em sẽ xây dựng mô tả tool rõ ràng như tài liệu API và theo dõi tỷ lệ lựa chọn công cụ chính xác mỗi khi thay đổi prompt hoặc mô hình.

Trần Khánh Hưng: Ban đầu em nghĩ chặn prompt injection chỉ cần lọc tin nhắn người dùng. Thực tế lọc đầu vào vẫn để 18,2% tấn công lọt qua dữ liệu công cụ trả về; quét thêm kết quả mới đưa về 0%, nhưng độ trễ p95 tăng từ 2,8 lên 5,7 giây. Khi làm thật, em sẽ dựng kịch bản tấn công trước, rồi chọn mức bảo vệ theo từng tool.

Tài liệu tham khảo

[1] OpenAI, "Function calling and other API updates", OpenAI Blog, 2023. https://openai.com/index/function-calling-and-other-api-updates/

[2] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models", ICLR, 2023. https://arxiv.org/abs/2210.03629

[3] T. Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools", NeurIPS, 2023. https://arxiv.org/abs/2302.04761

[4] Anthropic, "Introducing the Model Context Protocol", Anthropic News, 2024. https://www.anthropic.com/news/model-context-protocol

[5] K. Greshake et al., "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection", Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec '23), 2023, pp. 79–90. https://doi.org/10.1145/3605764.3623985

[6] OWASP, "OWASP Top 10 for LLM Applications 2025", OWASP GenAI Security Project, 2025. https://genai.owasp.org/llm-top-10/

[7] OpenAPI Initiative, "OpenAPI Specification v3.1.0", 2021. https://spec.openapis.org/oas/v3.1.0

[8] Klarna, "Klarna AI assistant handles two-thirds of customer service chats in its first month", Klarna Press Release, 2024. https://www.klarna.com/international/press/klarna-ai-assistant-handles-two-thirds-of-customer-service-chats-in-its-first-month/

[9] LinkedIn, "Hiring Assistant, LinkedIn’s first AI agent for recruiters, to launch globally in English", LinkedIn Pressroom, 03/09/2025. https://news.linkedin.com/2025/hiring-assistant-globally-available

[10] Computing, "Meat back on the menu as Klarna rows back on AI", Computing, 2025. https://www.computing.co.uk/news/2025/ai/meat-back-on-the-menu-as-klarna-rows-back-on-ai

[11] gRPC Authors, "Introduction to gRPC", gRPC Documentation, 2024. https://grpc.io/docs/what-is-grpc/introduction/